# Introduire le ViewModel

Commencer par une simple classe Dart

# Un ViewModel est d'abord une classe

```
class DashboardViewModel {
  final List<Sensor> sensors = [
    Sensor(
      id: 'living-room',
      name: 'Salon',
      room: 'Rez-de-chaussée',
      temperature: 21.8,
      humidity: 45,
      isOnline: true,
    ),
  ];
}
```

À ce stade, c'est une simple classe Dart, indépendante de la View.

Elle prépare les données dont la View a besoin.

# Une frontière simple

### DashboardViewModel

- conserve les données de l'écran ;
- expose des propriétés faciles à afficher ;
- calcule les informations dérivées.

### DashboardPage

- reçoit son ViewModel ;
- lit ses propriétés ;
- construit les widgets correspondants.

> Le ViewModel ne connaît ni widget, ni mise en page, ni `BuildContext`.

# Le parent crée, la View reçoit

```
void main() {
  final viewModel = DashboardViewModel();

  runApp(
    MaterialApp(
      home: DashboardPage(viewModel: viewModel),
    ),
  );
}
```

Le parent choisit et crée l'instance. `DashboardPage` se contente de la recevoir.

```
main() → DashboardViewModel → DashboardPage
```

Cette dépendance explicite permet de comprendre immédiatement avec quel objet la View travaille.

# La View reçoit son ViewModel

```
class DashboardPage extends StatelessWidget {
  const DashboardPage({
    super.key,
    required this.viewModel,
  });

  final DashboardViewModel viewModel;

  @override
  Widget build(BuildContext context) => SensorList(
    sensors: viewModel.sensors,
  );
}
```

```
DashboardPage(viewModel: DashboardViewModel())
```

La dépendance est explicite : la View affiche les données préparées par le ViewModel.

# La liste change de propriétaire

**Avant**

```
DashboardPage
├── List<Sensor>
├── calculs
└── widgets
```

**Après**

```
DashboardViewModel
├── List<Sensor>
└── propriétés calculées

DashboardPage
└── widgets
```

> Le résultat visuel ne change pas. Seule la responsabilité change de place.

# Des propriétés utiles à la View

```
class DashboardViewModel {
  final List<Sensor> _sensors = [/*  ...  */];

  List<Sensor> get sensors => List.unmodifiable(_sensors);
  int get sensorCount => _sensors.length;
  int get onlineCount =>
      _sensors.where((sensor) => sensor.isOnline).length;
  bool get hasOfflineSensors =>
      _sensors.any((sensor) => !sensor.isOnline);
}
```

- la liste devient privée ;
- la View peut la lire sans la modifier ;
- les valeurs dérivées sont calculées au même endroit ;
- la View reste concentrée sur l'affichage.

# Étape 2 — Extraire le ViewModel

- créer une classe `DashboardViewModel` ;
- déplacer la liste statique du widget vers cette classe ;
- rendre la liste privée et l'exposer en lecture seule ;
- ajouter `sensorCount`, `onlineCount` et `hasOfflineSensors` ;
- recevoir le ViewModel dans le constructeur de `DashboardPage` ;
- construire la liste avec `viewModel.sensors`.

## Terminé lorsque

Le dashboard affiche le même résultat, mais aucun capteur n'est déclaré dans la View.